F.7.03-03

Zhanat Umarova

Lectures Complex
of the discipline "Web services development (Java EE) "
for students of the EP 6B06120 "Information systems"

Shymkent 2026

MINISTRY OF EDUCATION AND SCIENCE OF THE REPUBLIC OF KAZAKHSTAN

M. AUEZOV SOUTH KAZAKHSTAN UNIVERSITY

Department of Information Systems and Modeling

Zhanat Umarova

Guidelines for implementation of the laboratory works

of the discipline "Web services development (Java EE)"

for students of the EP 6B06120 "Information systems"

Shymkent 2026

UDC 004.777

Compiled by: Umarova Zh.R., Guidelines for implementation of the laboratory works on the discipline "Web services development (Java EE)". - Shymkent: M.Auezov SKU, 2026 – 99 p.

The Guidelines for implementation of the laboratory works are compiled in accordance with the requirements of the curriculum and the program of the discipline " Web services development (Java EE)" and includes all the necessary theoretical information.

The Guidelines for implementation of the laboratory works contains the purpose of studying the topic, a plan, control questions and other necessary data. The Guidelines for implementation of the laboratory works topics is 15.

The Guidelines for implementation of the laboratory works is intended for students of the EP 6B06120 "Information systems".

Reviewers:

Botayeva S.B. – ctsc, associate professor, head of the department "Digital and information technologies", Tashenev University;

Suleimenova L.A.- ctsc, associate professor, head of the department “Computer science”

Considered and recommended for printing by the meeting of the Department of Information Systems and Modeling Protocol №__ from "__" _____ and the committee of the higher school of CE and E (Protocol №__ from "__" _____).

Recommended for publication by the Educational and Methodological Council of M.Auezov SKU protocol №___ dated "___" _____.

© M.Auezov South Kazakhstan University, 2026 y.

Responsible for edition is Zhanat Umarova

Laboratory course organization

The laboratory course uses one cumulative Course Registration Service. Each week extends the same Maven and Git repository so that students experience architecture, implementation, testing, deployment, and defense as one engineering process. Five laboratories are allocated five hours each and five laboratories are allocated four hours each, for a total of forty five hours.

Week	Laboratory work	Hours	Main deliverable
1	Create and inspect a Maven web application	5	Build a minimal Jakarta web application and explain the observed HTTP exchange.
2	Implement servlet endpoints filters and error handling	5	Create servlet routes, a filter chain, a listener, and centralized error handling.
3	Build and test CRUD REST endpoints	5	Implement a JAX RS resource with JSON DTOs and test the HTTP contract.
4	Map a relational domain model with JPA	5	Create entities, relationships, repositories, and transactional persistence operations.
5	Add validation exception mapping and OpenAPI	5	Enforce input constraints, return consistent errors, and document the API.
6	Refactor with CDI and transactional services	4	Separate endpoint, service, and repository responsibilities through dependency injection.
7	Protect endpoints and verify role based access	4	Apply authentication and authorization concepts and test access scenarios.
8	Consume an external service with resilience	4	Implement a REST client with timeouts, retry conditions, and safe failure behavior.
9	Create automated tests and containerized packaging	4	Build unit and integration tests and package the application reproducibly.
10	Profile document and defend the completed service	4	Add observability, measure selected behavior, document the service, and present evidence.

General submission requirements

- Source code in a Git repository with meaningful commits.
- A README containing prerequisites, build, run, test, and verification commands.
- Evidence of successful and unsuccessful cases: test output, HTTP exchanges, selected logs, or screenshots.
- A short report stating the goal, design choices, results, difficulties, and conclusion.
- No credentials, tokens, personal data, generated build folders, or IDE-specific secrets in the repository.

Assessment model

Criterion	Weight
Functional correctness	35%
Architecture and code quality	20%
Testing and evidence	20%

Criterion	Weight
Security and error handling	10%
Documentation and explanation	10%
Git discipline and timely submission	5%

Laboratory Work 1 Create and inspect a Maven web application

Week 1 Duration 5 hours

1 Topic explanation and purpose

Build a minimal Jakarta web application and explain the observed HTTP exchange. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

A maintainable service separates transport concerns from business rules and persistence. Resource or servlet code translates HTTP input into application commands, the service layer enforces use-case rules, and repositories isolate data access. This separation makes failures easier to locate and tests easier to design. In the context of create and inspect a maven web application, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	30 min
Instructor demonstration	35 min
Core implementation	120 min
Individual variant	55 min

Stage	Time
Testing and correction	40 min
Report and defense	20 min

6 Step by step procedure

1. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
2. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
3. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
4. Implement the normal path and verify it with one repeatable request or test.
5. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
6. Add an automated test or a documented verification script for both success and failure.
7. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
8. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
mvn archetype:generate -DgroupId=edu.example -DartifactId=course-service
+mvn clean package
+curl -i http://localhost:8080/course-service/health
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

9. Which component owns the lifecycle of the code created in this laboratory?
10. What is the public contract and how is a client expected to observe success?
11. Which invalid or failure cases are handled explicitly?
12. Which state is shared and what concurrency risk does it create?
13. Which resource must be closed, and by which owner?
14. How do the automated tests differ from the manual verification commands?
15. What security assumption was made and how could it fail?
16. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.
- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 2 Implement servlet endpoints filters and error handling

Week 2 Duration 5 hours

1 Topic explanation and purpose

Create servlet routes, a filter chain, a listener, and centralized error handling. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

The container owns component creation, lifecycle callbacks, concurrency, request dispatching, resource lookup, and deployment. Application code must respect these ownership boundaries: avoid unmanaged threads, close resources at the proper lifecycle point, and do not store request-specific mutable state in shared components. In the context of implement servlet endpoints filters and error handling, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	30 min
Instructor demonstration	35 min
Core implementation	120 min
Individual variant	55 min

Stage	Time
Testing and correction	40 min
Report and defense	20 min

6 Step by step procedure

17. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
18. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
19. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
20. Implement the normal path and verify it with one repeatable request or test.
21. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
22. Add an automated test or a documented verification script for both success and failure.
23. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
24. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
curl -i "http://localhost:8080/course-service/api/courses?limit=10"
+curl -i -X POST -d "title=Web Services" http://localhost:8080/course-service/courses
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

25. Which component owns the lifecycle of the code created in this laboratory?
26. What is the public contract and how is a client expected to observe success?
27. Which invalid or failure cases are handled explicitly?
28. Which state is shared and what concurrency risk does it create?
29. Which resource must be closed, and by which owner?
30. How do the automated tests differ from the manual verification commands?
31. What security assumption was made and how could it fail?
32. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.
- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 3 Build and test CRUD REST endpoints

Week 3 Duration 5 hours

1 Topic explanation and purpose

Implement a JAX RS resource with JSON DTOs and test the HTTP contract. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

Tests provide evidence at different boundaries. Unit tests isolate deterministic business rules, integration tests verify framework and database behavior, contract tests check API agreements, and deployment checks confirm the packaged artifact in its target environment. In the context of build and test crud rest endpoints, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	30 min
Instructor demonstration	35 min
Core implementation	120 min
Individual variant	55 min

Stage	Time
Testing and correction	40 min
Report and defense	20 min

6 Step by step procedure

33. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
34. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
35. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
36. Implement the normal path and verify it with one repeatable request or test.
37. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
38. Add an automated test or a documented verification script for both success and failure.
39. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
40. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
curl -i http://localhost:8080/course-service/api/courses
+curl -i -X POST -H "Content-Type: application/json" \
+  -d '{"code":"WEB101","title":"Web Services"}' \
+  http://localhost:8080/course-service/api/courses
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

41. Which component owns the lifecycle of the code created in this laboratory?
42. What is the public contract and how is a client expected to observe success?
43. Which invalid or failure cases are handled explicitly?
44. Which state is shared and what concurrency risk does it create?
45. Which resource must be closed, and by which owner?
46. How do the automated tests differ from the manual verification commands?
47. What security assumption was made and how could it fail?
48. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.

- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 4 Map a relational domain model with JPA

Week 4 Duration 5 hours

1 Topic explanation and purpose

Create entities, relationships, repositories, and transactional persistence operations. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

Persistence is not only object-to-table conversion. Identity, relationships, transaction boundaries, fetching, locking, database constraints, and failure recovery determine whether the application preserves invariants under concurrent access. In the context of map a relational domain model with jpa, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	30 min
Instructor demonstration	35 min
Core implementation	120 min
Individual variant	55 min
Testing and correction	40 min

Stage	Time
Report and defense	20 min

6 Step by step procedure

49. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
50. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
51. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
52. Implement the normal path and verify it with one repeatable request or test.
53. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
54. Add an automated test or a documented verification script for both success and failure.
55. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
56. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
@Entity
+class Enrollment {
+  @Id @GeneratedValue Long id;
+  @ManyToOne(optional=false) Student student;
+  @ManyToOne(optional=false) Course course;
+}
+mvn test
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

57. Which component owns the lifecycle of the code created in this laboratory?
58. What is the public contract and how is a client expected to observe success?
59. Which invalid or failure cases are handled explicitly?
60. Which state is shared and what concurrency risk does it create?
61. Which resource must be closed, and by which owner?
62. How do the automated tests differ from the manual verification commands?
63. What security assumption was made and how could it fail?
64. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.

- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 5 Add validation exception mapping and OpenAPI

Week 5 Duration 5 hours

1 Topic explanation and purpose

Enforce input constraints, return consistent errors, and document the API. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

HTTP is an application protocol built around requests and responses. Correct method semantics, status codes, headers, media types, caching rules, and conditional requests are part of the public behavior of a service. Treating HTTP merely as a tunnel produces APIs that are harder to test, cache, secure, and evolve. In the context of add validation exception mapping and openapi, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	30 min
Instructor demonstration	35 min
Core implementation	120 min
Individual variant	55 min

Stage	Time
Testing and correction	40 min
Report and defense	20 min

6 Step by step procedure

65. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
66. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
67. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
68. Implement the normal path and verify it with one repeatable request or test.
69. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
70. Add an automated test or a documented verification script for both success and failure.
71. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
72. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
public record CourseInput(
+  @NotBlank @Size(max=20) String code,
+  @NotBlank @Size(max=120) String title) {}
+
+curl -i -X POST -H "Content-Type: application/json" -d '{}' http://localhost:8080/course-service/api/courses
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

73. Which component owns the lifecycle of the code created in this laboratory?
74. What is the public contract and how is a client expected to observe success?
75. Which invalid or failure cases are handled explicitly?
76. Which state is shared and what concurrency risk does it create?
77. Which resource must be closed, and by which owner?
78. How do the automated tests differ from the manual verification commands?
79. What security assumption was made and how could it fail?
80. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.

- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 6 Refactor with CDI and transactional services

Week 6 Duration 4 hours

1 Topic explanation and purpose

Separate endpoint, service, and repository responsibilities through dependency injection. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

Persistence is not only object-to-table conversion. Identity, relationships, transaction boundaries, fetching, locking, database constraints, and failure recovery determine whether the application preserves invariants under concurrent access. In the context of refactor with cdi and transactional services, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	25 min
Instructor demonstration	30 min
Core implementation	95 min
Individual variant	40 min

Stage	Time
Testing and correction	30 min
Report and defense	20 min

6 Step by step procedure

81. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
82. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
83. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
84. Implement the normal path and verify it with one repeatable request or test.
85. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
86. Add an automated test or a documented verification script for both success and failure.
87. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
88. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
@ApplicationScoped
+public class EnrollmentService {
+  @Inject EnrollmentRepository repository;
+  @Transactional public Enrollment enroll(long studentId, long courseId) { /* enforce rules */ }
+}
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

89. Which component owns the lifecycle of the code created in this laboratory?
90. What is the public contract and how is a client expected to observe success?
91. Which invalid or failure cases are handled explicitly?
92. Which state is shared and what concurrency risk does it create?
93. Which resource must be closed, and by which owner?
94. How do the automated tests differ from the manual verification commands?
95. What security assumption was made and how could it fail?
96. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.

- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 7 Protect endpoints and verify role based access

Week 7 Duration 4 hours

1 Topic explanation and purpose

Apply authentication and authorization concepts and test access scenarios. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

A maintainable service separates transport concerns from business rules and persistence. Resource or servlet code translates HTTP input into application commands, the service layer enforces use-case rules, and repositories isolate data access. This separation makes failures easier to locate and tests easier to design. In the context of protect endpoints and verify role based access, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	25 min
Instructor demonstration	30 min
Core implementation	95 min
Individual variant	40 min

Stage	Time
Testing and correction	30 min
Report and defense	20 min

6 Step by step procedure

97. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
98. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
99. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
100. Implement the normal path and verify it with one repeatable request or test.
101. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
102. Add an automated test or a documented verification script for both success and failure.
103. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
104. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
@GET @Path("/{id}") @RolesAllowed({"student","teacher"})
+public CourseDto get(@PathParam("id") long id) { return service.get(id); }
+
+curl -i -H "Authorization: Bearer $ACCESS_TOKEN" http://localhost:8080/course-service/api/courses/1
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

105. Which component owns the lifecycle of the code created in this laboratory?
106. What is the public contract and how is a client expected to observe success?
107. Which invalid or failure cases are handled explicitly?
108. Which state is shared and what concurrency risk does it create?
109. Which resource must be closed, and by which owner?
110. How do the automated tests differ from the manual verification commands?
111. What security assumption was made and how could it fail?
112. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.

- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 8 Consume an external service with resilience

Week 8 Duration 4 hours

1 Topic explanation and purpose

Implement a REST client with timeouts, retry conditions, and safe failure behavior. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

A maintainable service separates transport concerns from business rules and persistence. Resource or servlet code translates HTTP input into application commands, the service layer enforces use-case rules, and repositories isolate data access. This separation makes failures easier to locate and tests easier to design. In the context of consume an external service with resilience, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	25 min
Instructor demonstration	30 min
Core implementation	95 min
Individual variant	40 min

Stage	Time
Testing and correction	30 min
Report and defense	20 min

6 Step by step procedure

113. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
114. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
115. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
116. Implement the normal path and verify it with one repeatable request or test.
117. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
118. Add an automated test or a documented verification script for both success and failure.
119. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
120. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
Client client = ClientBuilder.newBuilder()
+  .connectTimeout(2, SECONDS).readTimeout(3, SECONDS).build();
+try (Response r = client.target(baseUri).path("data").request().get()) {
+  if (r.getStatusInfo().getFamily() != SUCCESSFUL) throw new UpstreamException(r.getStatus());
+}
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

121. Which component owns the lifecycle of the code created in this laboratory?
122. What is the public contract and how is a client expected to observe success?
123. Which invalid or failure cases are handled explicitly?
124. Which state is shared and what concurrency risk does it create?
125. Which resource must be closed, and by which owner?
126. How do the automated tests differ from the manual verification commands?
127. What security assumption was made and how could it fail?
128. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.

- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 9 Create automated tests and containerized packaging

Week 9 Duration 4 hours

1 Topic explanation and purpose

Build unit and integration tests and package the application reproducibly. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

Tests provide evidence at different boundaries. Unit tests isolate deterministic business rules, integration tests verify framework and database behavior, contract tests check API agreements, and deployment checks confirm the packaged artifact in its target environment. In the context of create automated tests and containerized packaging, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	25 min
Instructor demonstration	30 min
Core implementation	95 min
Individual variant	40 min

Stage	Time
Testing and correction	30 min
Report and defense	20 min

6 Step by step procedure

129. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
130. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
131. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
132. Implement the normal path and verify it with one repeatable request or test.
133. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
134. Add an automated test or a documented verification script for both success and failure.
135. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
136. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
mvn clean verify
+docker build -t course-service:1.0 .
+docker run --rm -p 8080:8080 course-service:1.0
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

137. Which component owns the lifecycle of the code created in this laboratory?
138. What is the public contract and how is a client expected to observe success?
139. Which invalid or failure cases are handled explicitly?
140. Which state is shared and what concurrency risk does it create?
141. Which resource must be closed, and by which owner?
142. How do the automated tests differ from the manual verification commands?
143. What security assumption was made and how could it fail?
144. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.
- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.

Laboratory Work 10 Profile document and defend the completed service

Week 10 Duration 4 hours

1 Topic explanation and purpose

Add observability, measure selected behavior, document the service, and present evidence. This laboratory connects the weekly theoretical material to a reproducible implementation. Students must not only make the happy path work; they must also demonstrate an invalid, missing, forbidden, conflicting, or unavailable case appropriate to the topic.

A maintainable service separates transport concerns from business rules and persistence. Resource or servlet code translates HTTP input into application commands, the service layer enforces use-case rules, and repositories isolate data access. This separation makes failures easier to locate and tests easier to design. In the context of profile document and defend the completed service, students should identify the responsible component, the input and output contract, normal behavior, failure behavior, and evidence needed to verify the implementation.

2 Learning outcomes

- Configure or extend the cumulative Maven project without breaking previous functionality.
- Implement the required component and explain its lifecycle and responsibility.
- Verify the public behavior with repeatable commands or automated tests.
- Diagnose one deliberate failure from logs, status codes, exceptions, database state, or metrics.
- Document the result clearly enough for another student to reproduce it.

3 Required resources

Java 17 or later; Maven 3.9 or later; Git; a compatible Jakarta EE runtime or Apache Tomcat where appropriate; an IDE; curl or an API client; Docker for weeks 9 and 10; and the course repository. Use versions approved by the instructor and record them in the README.

4 Safety and preparation checks

- Create a feature branch and confirm that the previous week builds successfully.
- Do not commit passwords, access tokens, database files, or environment secrets.
- Use test data only. Limit retries, request sizes, and loops to prevent uncontrolled load.
- Record the exact command and expected result before changing configuration.

5 Work plan and time allocation

Stage	Time
Concept review and setup	25 min
Instructor demonstration	30 min
Core implementation	95 min
Individual variant	40 min

Stage	Time
Testing and correction	30 min
Report and defense	20 min

6 Step by step procedure

145. Update the feature branch and run mvn clean verify. Stop and resolve pre-existing failures before adding new code.
146. Review the requested HTTP or component contract. Write down the input, successful output, failure outputs, and observable side effects.
147. Create the smallest required class or configuration change. Keep HTTP, business, and persistence responsibilities separated.
148. Implement the normal path and verify it with one repeatable request or test.
149. Implement validation and error behavior. Ensure the response does not expose stack traces or secrets.
150. Add an automated test or a documented verification script for both success and failure.
151. Review logs and resource cleanup. Confirm that connections, responses, clients, and containers are closed by the proper owner.
152. Commit the implementation and update the README and laboratory report.

7 Example of implementation

Illustrative example

```
curl -i http://localhost:8080/health/ready
+curl -i "http://localhost:8080/course-service/api/courses?page=0&size=20"
+mvn clean verify
```

The example shows the minimum direction, not a complete answer. Students must adapt package names, deployment context, configuration, data model, and error representation to the cumulative project. Every command included in the report must be executed and its actual result recorded.

8 Required verification

Case	Expected evidence
Successful case	Correct status or component result, expected body or state, and no unexpected error log.
Invalid input	Controlled 400-level response or validation result with a useful field-level message.
Missing or denied resource	Appropriate 404 or 403 result without information leakage.
Repeat or concurrency case	Stable behavior for a repeated request, duplicate submission, or concurrent access where relevant.
Build and regression	mvn clean verify succeeds and previous endpoints still behave as documented.

9 Individual assignments

Variant	Individual extension

Variant	Individual extension
1	Add a search or filter parameter with validation, deterministic ordering, and tests.
2	Add a second resource type and model the relationship without duplicating business logic.
3	Add a conflict or duplicate case and return a consistent error representation.
4	Add pagination or a bounded collection response and document limits.
5	Add an audit field or correlation identifier and verify it through logs or response headers.
6	Add a role or ownership rule and test allowed and denied scenarios.
7	Add a timeout or controlled infrastructure failure and demonstrate recovery behavior.
8	Add an integration test using an isolated database or container.
9	Add an OpenAPI example and check that implementation and documentation agree.
10	Measure one operation, establish a baseline, make one justified improvement, and compare results.

10 Report structure

- Title, student, group, date, and selected variant.
- Purpose and expected learning outcome.
- Architecture or request flow, with a short explanation.
- Implementation: key classes and configuration, without copying the entire project.
- Verification table containing command or test, expected result, actual result, and conclusion.
- Problems encountered and how evidence led to the correction.
- Final conclusion and link or commit identifier.

11 Control questions

153. Which component owns the lifecycle of the code created in this laboratory?
154. What is the public contract and how is a client expected to observe success?
155. Which invalid or failure cases are handled explicitly?
156. Which state is shared and what concurrency risk does it create?
157. Which resource must be closed, and by which owner?
158. How do the automated tests differ from the manual verification commands?
159. What security assumption was made and how could it fail?
160. Which evidence proves that the result is reproducible?

12 Assessment checklist

- Required endpoint or component works.
- Success and failure cases are demonstrated.
- Previous functionality remains operational.
- Code responsibilities are separated and names are clear.
- Tests and report contain actual evidence.
- Selected individual variant is complete.
- Student can explain design choices without reading the report verbatim.

13 Literature

M: 1–9; A: 10–13. Use the specifications corresponding to the weekly topic and the official documentation of the selected runtime and tools.